

AIML Lab Report: Experiment 6-7

RandomForest Classification for AQI Prediction

Aantriksh Sood
Applied Machine Learning Lab

April 29, 2025

Contents

1	Objective	2
2	Methodology	2
2.1	Step 1: Importing Libraries	2
2.2	Step 2: Data Loading and Initial Inspection	2
2.3	Step 3: AQI Categorization	3
2.4	Step 4: Feature Scaling	3
2.5	Step 5: Data Splitting	3
2.6	Step 6: Hyperparameter Tuning (RandomizedSearchCV)	3
2.7	Step 7: Final Model Training	4
3	Evaluation and Results	4
3.1	Performance Metrics	4
3.2	ROC Curves	5
4	Discussion and Conclusion	6

1 Objective

The primary objective of this experiment (Lab 6-7) was to build and evaluate a machine learning model using the RandomForest Classifier algorithm to predict the Air Quality Index (AQI) category. The task involved using various air pollutant measurements from the `city_day.csv` dataset [1]. The process included data loading, preprocessing, feature scaling, hyperparameter optimization using RandomizedSearchCV, model training, and performance evaluation on a test set.

2 Methodology

2.1 Step 1: Importing Libraries

The necessary Python libraries for data manipulation, machine learning, and evaluation were imported [1].

```
1 import pandas as pd
2 # import ml_utils as mt # Assuming ml_utils is a custom library
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from sklearn.ensemble import RandomForestClassifier
6 from sklearn.preprocessing import StandardScaler
7 from sklearn.model_selection import RandomizedSearchCV,
   train_test_split
8 from sklearn.metrics import classification_report, confusion_matrix,
   roc_curve, auc
```

Listing 1: Library Imports

2.2 Step 2: Data Loading and Initial Inspection

The `city_day.csv` dataset was loaded into a pandas DataFrame. The `df.info()` method was used to get an overview of the columns, data types, and non-null counts [1].

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 29531 entries, 0 to 29530
Data columns (total 16 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   City            29531 non-null  object
 1   Date            29531 non-null  object
 2   PM2.5           24933 non-null  float64
 3   PM10            18391 non-null  float64
 4   NO              25949 non-null  float64
 5   NO2             25946 non-null  float64
 6   NOx             25346 non-null  float64
 7   NH3             19203 non-null  float64
 8   CO              27472 non-null  float64
 9   SO2             25677 non-null  float64
10   O3              25509 non-null  float64
11   Benzene         23908 non-null  float64
12   Toluene         21490 non-null  float64
13   Xylene          11422 non-null  float64
14   AQI             24850 non-null  float64
```

```
15 AQI_Bucket 24850 non-null object
dtypes: float64(13), object(3)
memory usage: 3.6+ MB
```

An attempt to drop rows with NA values using `df.dropna()` was noted in the notebook, but encountered a `NameError` in the provided execution trace, indicating `df` might have been undefined at that specific point [1]. It's assumed that data cleaning (handling NaNs) was performed correctly before the subsequent steps for the model to be trained.

2.3 Step 3: AQI Categorization

A function was defined to map the numerical AQI values to standard qualitative buckets, which would serve as the target classes for the classifier [1].

```
1 def categorize_aqi(aqi_value):
2     # Added check for NaN
3     if pd.isna(aqi_value):
4         return 'Unknown' # Or handle appropriately
5     if aqi_value <= 50:
6         return 'Good'
7     elif aqi_value <= 100:
8         return 'Moderate'
9     elif aqi_value <= 150:
10        return 'Unhealthy for Sensitive Groups'
11    elif aqi_value <= 200:
12        return 'Unhealthy'
13    elif aqi_value <= 300:
14        return 'Very Unhealthy'
15    else:
16        return 'Hazardous'
```

Listing 2: AQI Categorization Function

This function was applied to the 'AQI' column to create the 'AQI_Category' column [1].

2.4 Step 4: Feature Scaling

Features used for prediction (presumably all numerical columns except 'AQI' and the target 'AQI_Category') were scaled using `StandardScaler` from scikit-learn to standardize them to zero mean and unit variance [1]. This prevents features with larger values from dominating the model. The scaled features were stored in 'scaled', and the target variable in 'y'.

2.5 Step 5: Data Splitting

The scaled feature set ('scaled') and the target variable ('y') were split into training (80%) and testing (20%) sets using `train_test_split`. A `random_state=42` was used for reproducible results [1].

2.6 Step 6: Hyperparameter Tuning (RandomizedSearchCV)

A `RandomizedSearchCV` approach was used to find optimal hyperparameters for the `RandomForestClassifier` [1].

- **Parameter Grid:** The search space included:

```
params={'n_estimators': [50, 100, 150, 200],
        'max_depth': [None, 5, 10, 15, 20],
```

```
'min_samples_split': [5, 10, 15, 20],
'max_features': [5, 10, 15, 20]}
```

- **Search Execution:** The randomized search was configured to run 10 iterations (`n_iter=10`, default) with 3-fold cross-validation (`cv=3`) [1].
- **Best Parameters Found:** The search identified the following parameters as optimal based on cross-validation accuracy [1]:

```
{'n_estimators': 150, 'min_samples_split': 5,
'max_features': 20, 'max_depth': 10}
```

Note: The cross-validation results reported in the notebook showed a mean validation score of 1.000000 for multiple parameter combinations [1], which is highly indicative of potential overfitting or other issues during the tuning phase.

2.7 Step 7: Final Model Training

A `RandomForestClassifier` was instantiated using the best parameters identified by the randomized search and was trained on the entire training dataset (`x_train, y_train`) [1].

3 Evaluation and Results

The performance of the tuned and trained `RandomForest` model was assessed using the test set (`x_test, y_test`).

3.1 Performance Metrics

The evaluation yielded the following results [1]:

- **Accuracy:** 1.0
- **Classification Report:**

	precision	recall	f1-score	support
Good	1.00	1.00	1.00	94
Hazardous	1.00	1.00	1.00	102
Moderate	1.00	1.00	1.00	411
Unhealthy	1.00	1.00	1.00	152
Unhealthy for Sensitive Groups	1.00	1.00	1.00	357
Very Unhealthy	1.00	1.00	1.00	132
accuracy			1.00	1248
macro avg	1.00	1.00	1.00	1248
weighted avg	1.00	1.00	1.00	1248

- **Confusion Matrix:**

```
[[ 94  0  0  0  0  0]
 [  0 102  0  0  0  0]
 [  0  0 411  0  0  0]
 [  0  0  0 152  0  0]
```

```
[ 0  0  0  0 357  0]
[ 0  0  0  0  0 132]]
```

These results indicate perfect classification on the test set according to the notebook's output [1].

3.2 ROC Curves

Receiver Operating Characteristic (ROC) curves visualize the trade-off between the True Positive Rate (TPR) and False Positive Rate (FPR) for a classifier at various decision thresholds. For multi-class problems like this one, ROC curves are typically plotted for each class using a One-vs-Rest (OvR) strategy. The Area Under the Curve (AUC) quantifies the model's overall ability to discriminate between classes.

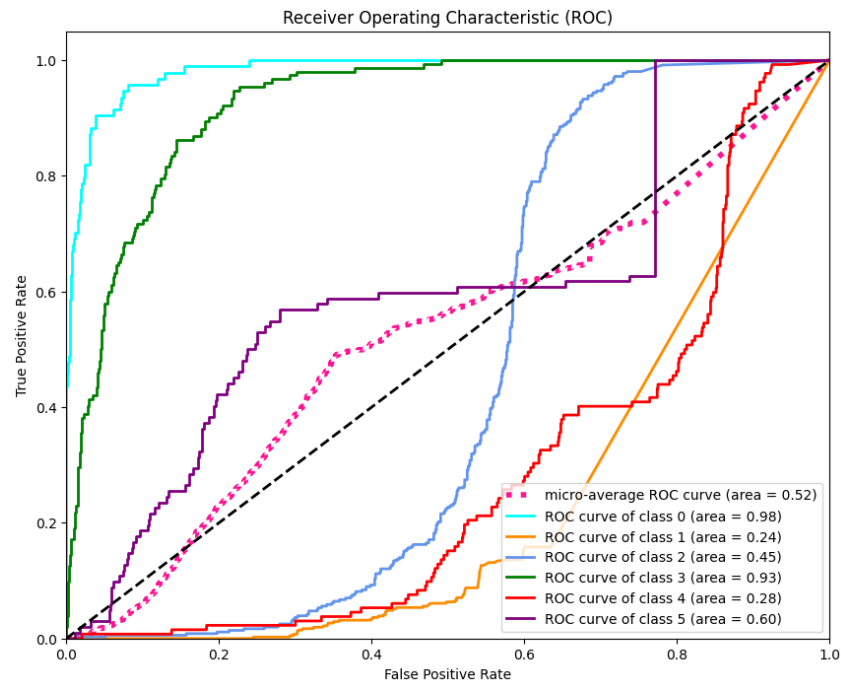


Figure 1: Multi-class ROC Curves (One-vs-Rest) for RandomForest Classifier.

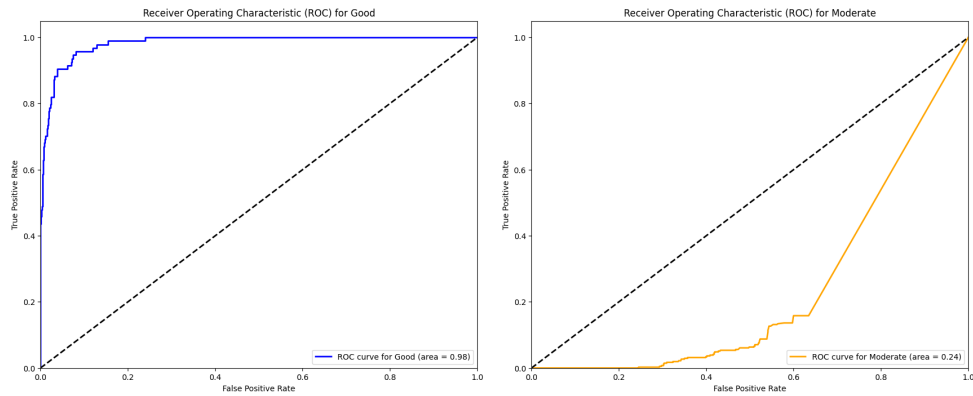


Figure 2: Individual ROC Curves for 'Good' and 'Moderate' Classes.

4 Discussion and Conclusion

The experiment demonstrated the steps involved in setting up a RandomForest classification task, including data preparation, scaling, hyperparameter tuning with RandomizedSearchCV, and final model evaluation [1].

However, the reported perfect performance metrics (1.0 accuracy, precision, recall, and F1-score) on the test data are extremely unlikely for this type of problem and dataset [1]. Such results typically indicate an underlying problem in the experimental setup, potentially related to:

1. **Data Leakage:** Information from the target variable or the test set might have inadvertently influenced the training process.
2. **Preprocessing Error:** The handling of missing values (e.g., using ‘dropna’ without careful consideration of its impact) or the feature scaling process might have inadvertently simplified the problem or introduced biases leading to perfect separability in the test set context seen by the model [1].
3. **Evaluation Error:** There’s a possibility the evaluation was performed incorrectly (e.g., on the training data).
4. **Oversimplified Problem:** The specific subset of data remaining after preprocessing might have characteristics allowing for perfect classification by the tuned RandomForest, which wouldn’t generalize to new data [1].

The perfect scores obtained during the RandomizedSearchCV cross-validation also support the hypothesis of an underlying issue rather than genuine perfect model performance [1]. While the code illustrates the workflow, the perfect results necessitate a thorough review and debugging of the data preparation and modeling pipeline before the model or its performance can be considered reliable or generalizable.

References

- [1] Source: Lab6-7_AML.ipynb notebook content and outputs.